CPSC 335 Final Exam Study Guide

Chapters 8–14 + Parallel Algorithms
Based on Kevin A. Wortman lecture slides · CSU Fullerton

Closed-book exam. One sheet of notes allowed (both sides). Bring your CWID. Not on exam: American Change Making; Ch. 12.6 reduction lower-bound arguments; parallel merge details. You will **not** be asked to prove an NP-complete reduction.

CPSC 335 Final Exam Study Guide

Chapters 8–14 + Parallel Algorithms (Kevin A. Wortman)

Exam format: Written, closed-book, closed-Internet, timed, non-cumulative. Mixture of multiple choice, short answer, and long answer. Bring your **CWID**. You may use **one sheet** of notes (8.5×11" or A4, both sides, handwritten or typed); turn it in with the exam.

What you may be asked to do: define terminology; write a problem definition; perform a step count; prove an efficiency class; analyze pseudocode (step count + efficiency proof); **trace** algorithms; **design** algorithms using decrease-by-half, reduction, and dynamic programming; justify lower bounds; categorize problems into **P** / **NP-complete** / **undecidable**; prove a lower bound; prove a problem is in **P** or **NP**. You will **not** be asked to prove an NP-complete reduction.

Struck-through / not on exam: American Change Making (11.3); Reduction Arguments for lower bounds (12.6); **parallel merge** (but parallel merge sort and samplesort are in scope).

Quick Reference: Must-Know Complexities

Topic	Result
Merge Sort	**$O(n \log n)$** deterministic
Out-of-place / In-place Quick Sort	**$O(n \log n)$** expected, **$O(n^2)$** worst-case
Hash table critical ops	**$O(1)$** expected
Naïve → hash reduction (set intersect, mode, …)	**$O(n^2)$** → **$O(n)$** expected
Prim–Jarník / Dijkstra with PSQ	**$O(m + n \log n)$**
Fibonacci DP	**$O(n)$**
Edit Distance DP	**$O(n^2)$**
Comparison sorting lower bound	**$\Omega(n \log n)$** → tight **$\Theta(n \log n)$**
Parallel sorting	**$O(n \log n)$** work, **sublinear** span
Parallel merge sort span	**$O(\log^3 n)$**
Samplesort span	**$O(\log n)$** expected

Chapter 8 — Decrease-by-Half (Divide and Conquer)

8.1 The Big Idea

Decrease-by-half splits the input into two halves, recursively solves each half, then **combines** the solutions.

```
decrease_by_half(INPUT):
  if INPUT is base case:
      return base case solution
  left_input, right_input = divide INPUT in half
  left_solution  = decrease_by_half(left_input)
  right_solution = decrease_by_half(right_input)
  entire_solution = combine left_solution with right_solution
  return entire_solution
```

Three phases: **Divide → Recursion → Combine**.

Base-case best practice: when dividing a list in half, handle **n ≤ 1** as base cases; general case is **n ≥ 2**. Missing the n=1 base case can cause **infinite recursion** (termination failure).

Dividing a list:

```
half = n / 2          # integer truncation
left  = L[:half]      # n/2 steps
right = L[half:]      # n/2 steps
```

8.2 Example: Summation

Problem: input list L of numbers; output sum of elements.

```
dbh_sum(L):
  n = len(L)
  if n == 0: return 0
  if n == 1: return L[0]
  half = n / 2
  left  = L[:half]
  right = L[half:]
  return dbh_sum(left) + dbh_sum(right)
```

Combine phase = **add** the two recursive results.
Analysis: T(n) = 2T(n/2) + n + 4 → **O(n log n)**.
(Note: naïve sum is **O(n)** and usually better — decrease-by-half is not always the right tool.)

8.3 Master Method (prove efficiency of decrease-by-fraction algorithms)

Master-form recurrence

[
T(n) = r \cdot T(n/d) + c(n) \quad \forall n \ge t
]
[
T(n) \in O(1) \quad \forall n < t
]
with (c(n) \in O(n^k)), and constants (r,d,t,k) where (r,t \ge 1), (d > 1), (k \ge 0).

Symbol	Meaning
r	number of recursive calls
d	divisor of input size
t	general-case threshold (smallest n that takes the general case)
k	exponent: non-recursive work is $O(n^k)$
c(n)	time except recursion

Master Theorem

Given a master-form recurrence:
- **Case 1:** if (r < d^k), then (T(n) \in O(n^k))
- **Case 2:** if (r = d^k), then (T(n) \in O(n^k \log n))
- **Case 3:** if (r > d^k), then (T(n) \in O(n^{\log_d r}))

Master Method steps (memorize this checklist)

1. Identify general-case threshold **t**
2. Analyze **base case** (n < t) → usually O(1)
3. Step-count **general case** (n ≥ t) → get recurrence T(n)
4. Identify / check master-form parts: c(n), r, d, t, k
5. Compare **r $\stackrel{?}{=}$ d^k**, identify case
6. Simplify and state conclusion

Worked example (dbh_sum / merge_sort style)

- t = 2; base: T(n) ∈ O(1) ∀ n < 2
- General: T(n) = 2T(n/2) + Θ(n) ∀ n ≥ 2
- r=2, d=2, k=1 → r = d^k → **Case 2** → **O(n log n)**

Binary search example

- T(n) = T(n/2) + O(1); r=1, d=2, k=0 → r = d^k → Case 2 → **O(log n)**

8.4 Merge Sort

Sorting problem: input list U of comparable elements; output list S with elements of U in non-decreasing order.

```
merge_sort(U):
  if n ≤ 1: return U
  half = n / 2
  left  = U[:half]
  right = U[half:]
  return merge(merge_sort(left), merge_sort(right))

merge(LS, RS):
  S = []; li = 0; ri = 0
  while li < len(LS) and ri < len(RS):
      if LS[li] <= RS[ri]:
          S.add_back(LS[li]); li += 1
      else:
          S.add_back(RS[ri]); ri += 1
  return S + LS[li:] + RS[ri:]
```

- **merge** takes **O(n)** (two-finger scan of already-sorted halves)
- **merge_sort** by Master Method: T(n)=2T(n/2)+O(n) → **O(n log n)**
- Be ready to **trace** on a small list: show every sub-list and every merge output
- Be ready to **design similar** decrease-by-half algorithms

Chapter 9 — Randomization

9.1 The Big Idea

Randomization = intentionally making random choices. Helps when there are many alternatives, most are good, and selecting the best would be slow.

- **Deterministic:** no randomness (all prior algorithms)
- **Randomized:** uses random choices

9.2 Generating Random Numbers

Operation	Pseudocode	Time
Random int in [min, max]	`random_int(min, max)`	O(1)
Random element of L	`random_choice(L)`	O(1)
Shuffle L	`random_shuffle(L)`	O(n)

- **TRNG:** true random (physical entropy)
- **PRNG:** deterministic formula that looks random (default meaning of "random" in this course)

Expected time

- **Expected value** E[X] = weighted average of outcomes
- **O(f(n)) expected** means E[runtime] ∈ O(f(n))
- Preference: deterministic > amortized > expected
- "Expected" is sticky: O(x) expected + O(y) deterministic = O(x+y) **expected**

9.4 The Las Vegas Pattern

A **Las Vegas** algorithm always returns a **correct** answer; randomness only affects **runtime** (may be slow with small probability). Randomized quick sort is Las Vegas: always sorts correctly; expected O(n log n), worst-case $O(n^2)$.

9.5 Out-of-Place Quick Sort

Key difference from merge sort: divide by **value** (pivot/partition), not by index. Combine is cheap (append).

```
quick_sort(U):
  if n ≤ 1: return U
  pivot = random_choice(U)          # Las Vegas / randomization
  less    = [x for x in U if x <  pivot]
  equal   = [x for x in U if x == pivot]
  greater = [x for x in U if x >  pivot]
  return quick_sort(less) + equal + quick_sort(greater)
```

- `equal` needs **no** recursive sort (already tied / non-decreasing)
- Combine = **append** three lists (every LS element < equal < every GS element)
- Balanced pivots → Master Form → **O(n log n)**
- Extreme pivots (always min/max) → T(n)=T(n−1)+O(n) → **O(n²)**
- Random pivot → **O(n log n) expected**, **O(n²) worst-case**
- Be able to **trace by hand**

Deterministic (pivot = U[0]) can hit O(n²) on sorted input — that is why we randomize.

9.6 In-Place Quick Sort

Same efficiency as out-of-place: **O(n log n) expected**, **O(n²) worst-case**, but **in-place** (rearranges U; better space / constants).

```
in_place_quick_sort(U):
  quick_sort_range(U, 0, len(U))
  return U

quick_sort_range(U, s, e):
  if (e - s) ≤ 1: return
  p = in_place_partition(U, s, e)
  quick_sort_range(U, s, p)
  quick_sort_range(U, p + 1, e)

in_place_partition(U, s, e):
  swap(U[random_int(s, e-1)], U[e-1])   # random pivot → end
  pivot_value = U[e-1]
  i = s; j = e - 2
  while i <= j:
      if U[i] < pivot_value: i++
      else if U[j] >= pivot_value: j--   # (slides use ≤ in one place; idea: grow ≥ zone)
      else: swap(U[i], U[j]); i++; j--
  p = i
  swap(U[e-1], U[p])
  return p
```

Zones during partition: `[s:i)` < pivot | todo | `[j+1:e-1]` ≥ pivot | `U[e-1]` = pivot.
Final swap places pivot between less and greater. Partition itself is **O(n)**.

Sorting comparison (from slides)

Algorithm	Expected	Worst-case	In-place?
Selection Sort	$O(n^2)$	$O(n^2)$	yes
Merge Sort	**$O(n \log n)$**	**$O(n \log n)$**	no
Quick Sort	**$O(n \log n)$**	$O(n^2)$	**yes**

Default: Quick Sort. If expected time is unacceptable: Merge Sort. Never use $O(n^2)$ sorts.

Chapter 10 — Reduction

10.1 The Big Idea

Reduction: use an off-the-shelf algorithm or data structure to do the hard part.

Reduction to an algorithm (template):

```
reduction_pattern(input_A):
  input_B = pre-process input_A
  solution_B = solve_B(input_B)
  solution_A = post-process solution_B
  return solution_A
```

If A reduces to B, then A is **easier than (or tied with)** B.

Informal analysis (allowed after midterm): cite big-O of bottleneck steps only; skip full step-count proofs when obvious.

10.2 Reduction to Sorting

Template:

```
reduction_to_sorting(input_A):
  unsorted = pre-process input_A
  sorted_list = quick_sort(unsorted)
  return post-process(sorted_list)
```

Usually **O(n log n)** expected (dominated by sort).

Median finding

Sort U, return element at index ⌊n/2⌋ (or appropriate middle). **O(n log n)** expected via reduction to sorting.

Set intersection (via sorting)

```
set_intersect_sort(L, R):
  sorted = quick_sort(L + R)
  output = []
  for i from 0 to len(sorted) - 2:
      if sorted[i] == sorted[i+1]:
          output.add(sorted[i])
  return output
```

Idea: concatenate → sort → duplicates are the intersection. **O(n log n)** expected.

Min and max (via sorting)

Sort, return (first, last). **O(n log n)** expected. (A linear scan is faster in practice; this is a reduction example.)

10.3 Reduction to Hash Table Operations

How hash tables work (overview)

- **Map:** stores **associations** (key → value)
- **Hashing:** key → hash code → compressed array index
- **Collisions** inevitable (pigeonhole principle); handled by **chaining** or open addressing
- Critical ops: **O(1) expected** (`M[k]`, `k in M`; insert/delete also amortized expected)

Pattern: speed $O(n^2)$ naïve → O(n) expected

1. Write naïve algorithm
2. Find **bottleneck** (O(n) search/count inside an O(n) loop)
3. Replace with **O(1) expected** hash ops (pre-populate the table)

Set intersection via hash:

```
set_intersect_hash(L, R):
  hm = HashMap()
  for r in R: hm[r] = True
  S = []
  for x in L:
      if x in hm: S.add(x)
  return S
```

O(n) expected (was $O(n^2)$ naïve).

Mode average via hash: count frequencies in a map, then scan for max count → **O(n) expected**.

10.4 Priority Search Queues (Prim–Jarník & Dijkstra)

PSQ ADT (Fibonacci-heap style times from slides):
Op	Meaning	Time
Insert(key, p)	insert with priority	O(1)
RemoveMin()	remove most urgent (smallest priority)	O(log n) amortized
DecreasePriority(key, p)	make more urgent	O(1) amortized

Prim–Jarník MST and **Dijkstra NNSSSPP** with a PSQ: **O(m + n log n)**
(without PSQ: O(mn)).

Correspondence idea:
- Prim: keys = edges; priority = weight if bridge, else ∞
- Dijkstra: keys = vertices; priority = known distance, else ∞

Know: reducing the “find min bridge / closest unseen” sequential search to PSQ RemoveMin / DecreasePriority yields **O(m + n log n)**.

Chapter 11 — Dynamic Programming

11.1 The Big Idea

Tabulation: store solutions to problem instances in a **table**.
- Base cases: table elements initialized immediately
- General cases: initialized from other table elements
- Algorithm initializes **all** table elements; finally the solution is in the table

Top-down thinking = recursion / decrease-by-half (break large → small).
Bottom-up thinking = dynamic programming (combine small → large).

Design process

1. Choose dimensions (1D, 2D, …)
2. Write a **table invariant** (what each `A[i]` / `A[i][j]` stores)
3. Base-case pseudocode
4. General-case pseudocode
5. Draft complete bottom-up algorithm

1D Template

```
dynamic_programming_1D(input):
  A = array large enough
  Initialize base-case elements of A
  for i from smallest general case to largest general case:
      A[i] = initialize using elements A[<i]
  return A[solution index]
```

~~11.3 American Change Making — NOT ON EXAM~~

11.2 1D DP — Fibonacci Numbers (O(n))

Fibonacci Number Problem: input integer (n \ge 0); output (F(n)).

[
F(0)=0,\quad F(1)=1,\quad F(n)=F(n-1)+F(n-2)\ (n\ge 2)
]
First ten: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34.

Naïve recursion is exponential (two recursive calls; overlapping subproblems). Extremely slow.

Table invariant: `A[i] = F(i)` (1D table).

```
fibonacci_dynamic_programming(n):
  A = [0] * (n + 1)
  A[1] = 1
  for i from 2 to n:
      A[i] = A[i-1] + A[i-2]
  return A[n]
```

Analysis: initializing A is O(n); loop is O(n) → **O(n)** total. Huge speedup vs exponential recursion.

Be able to **trace** (show table A) and **design similar 1D DP**.

Edit Distance (O(|s| × |t|) / O(n²))

Edit Distance Problem: input start string `s` and goal string `t`; output the edit distance between them.

Levenshtein edit distance = minimum operations to transform `s` into `t`:
1. **Overwrite** (substitute) a character
2. **Insert** a character
3. **Delete** a character

Example: `"dikestra"` → `"dijkstra"` needs insert `j` + delete `e` → distance **2**.

Dimensions: two ways to shrink input (shrink `s`, shrink `t`) → **2D table**.

Table invariant:
`A[i][j]` = edit distance between `s[:i]` and `t[:j]`
(= min ops to transform the first `i` chars of `s` into the first `j` chars of `t`).

Base cases:
- `A[0][j] = j` (transform `""` into `t[:j]` by `j` inserts)
- `A[i][0] = i` (transform `s[:i]` into `""` by `i` deletes)

General case (four alternatives):

```
if s[i-1] == t[j-1]:
    A[i][j] = A[i-1][j-1]          # match: no action
else:
    A[i][j] = min(
        1 + A[i-1][j],              # delete
        1 + A[i][j-1],              # insert
        1 + A[i-1][j-1])            # overwrite
```

Final algorithm:

```
edit_distance(s, t):
  A = array[s.size+1][t.size+1] initialized to 0
  for i from 0 to s.size: A[i][0] = i
  for j from 0 to t.size: A[0][j] = j
  for j from 1 to t.size:
      for i from 1 to s.size:
          if s[i-1] == t[j-1]:
              A[i][j] = A[i-1][j-1]
          else:
              A[i][j] = min(1+A[i-1][j], 1+A[i][j-1], 1+A[i-1][j-1])
  return A[s.size][t.size]
```

Analysis: nested loops over |s| and |t| → **O(|s| × |t|)** = **O(n²)** when lengths are Θ(n).

Be able to **trace a full table** (slide demo: `s="has"`, `t="make"`) and design similar 2D DP.

Chapter 12 — Lower Bounds

12.1 The Big Idea (Act II)

Act I proved things about **algorithms**. Act II proves things about **problems**.

Limitation	Meaning	Software impact
Lower bound	“speed limit”	must live with that runtime
NP-complete	apparently needs exp. exhaustive search	too slow to use
Undecidable	impossible to solve	can never implement

12.2 Notation

- **Upper bound** for a problem: established by exhibiting an algorithm (proof by construction). Example: merge sort ⇒ sorting has upper bound **O(n log n)**.
- **Lower bound Ω(f(n)):** every algorithm solving the problem takes at least that long. Hard — “for all algorithms,” cannot prove by construction.
- **Tight bound Θ(f(n)):** upper and lower bounds **match** → algorithm design for that problem is “done / solved science.”

Intuition:
- **O(f)** = “at most f”
- **Ω(f)** = “at least f”
- **Θ(f)** = “asymptotically equal to f”

12.3 Proving Negatives

Proving a positive is easy (show an example). Proving a negative (“no algorithm is faster than…”) is hard. That is why lower-bound proofs feel different.

12.4 Trivial Lower Bounds

Strategy: every correct algorithm must create the output → time at least the output size.

Merge example: output has n elements ⇒ Merge has lower bound **Ω(n)**.
Combined with merge’s O(n) algorithm ⇒ Merge has tight bound **Θ(n)**.

Sorting trivial lower bound: output has n elements ⇒ **Ω(n)** — but this leaves a gap below the O(n log n) upper bound.

12.5 The Tight Bound for Sorting

Lemma: Comparison-based sorting has lower bound **Ω(n log n)**.

Proof sketch (information-theoretic):
1. n distinct elements ⇒ **n!** possible permutations
2. Each comparison of a vs b eliminates about **half** the remaining permutations

3. Need (\log_2(n!)) comparisons to isolate one permutation
4. (\log_2(n!) = \sum_{i=1}^{n} \log_2 i); there are n terms that are Ω(log n) each
5. So (\log_2(n!) \in \Omega(n \log n))
6. Therefore every comparison-based sorting algorithm takes **Ω(n log n)** time

Theorem: Sorting has tight bound **Θ(n log n)**
(Ω from lemma + O from merge sort).

Status: Solved science — no comparison-based sort can be asymptotically faster than O(n log n).

~~12.6 Reduction Arguments — NOT ON EXAM~~
(Do not spend exam-prep time on partial-sort reduction lower-bound proofs.)

Chapter 13 — Easy and Impossible Problems

13.1 / 13.2 Efficiently Solvable Problems and P

Complexity class = a set of problems.

P = { X | X can be solved in **polynomial time** }

Polynomial time: time $\in O(n^k)$ for some constant $k \geq 0$.
Includes $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$.
Not polynomial: $O(2^n)$, $O(n!)$.

(Lemma: $\log n \in O(n)$ and $n \log n \in O(n^2)$, so log factors still count as polynomial.)

Prove X ∈ P (template)

1. Design any polynomial algorithm for X (naïve is fine)
2. Analyze → show polynomial time
3. Conclude X ∈ P

```
Theorem: Problem X ∈ P.
Proof: The following algorithm solves X:
  <pseudocode>
This takes O(TIME) time, which is polynomial. QED
```

Example: Pair Sum naïve double loop is $O(n^2) \Rightarrow$ Pair Sum $\in$ P.

Problems in P from the course include sorting, merge, edit distance, Fibonacci, MST, Dijkstra/NNSSSPP, set intersection, etc.

13.3 Unsolvable Problems and Decidability

- **Decidable:** some algorithm solves it (any finite time)
- **Undecidable: no** algorithm solves it

P ⊆ Decidable.

The Halting Problem

Input: procedure `proc` and compatible input `I`
Output: True if `proc(I)` **halts**; False if it **hangs** (loops forever)

Theorem: Halting is undecidable (proof by contradiction / diagonalization):
1. Suppose `halts(proc, I)` exists
2. `self_halts(proc) = halts(proc, proc)`
3. Build `contrary(proc)`: if `self_halts(proc)` then loop forever, else return
4. Evaluate `contrary(contrary)` → both cases contradict
5. Therefore `halts` cannot exist

Implication: compilers cannot reliably detect all infinite loops.

Chapter 14 — NP-Completeness / NP-Hard Problems

14.1 Verifiable Problems and NP

NP = { X | exhaustive algorithm solves X, and the **verifier** runs in polynomial time }.

Solve = turn input into output. **Verify** = check one **candidate**.

Prove X ∈ NP (template)

1. Define a candidate
2. Design the verifier
3. Prove verifier is polynomial time
4. Conclude X ∈ NP

Corollary: P ⊆ NP.

14.2 Hard Problems and NP-Hardness

Polynomial-time reduction: (E \le_p H) means there is a reduction solving E by calling a solver for H, with poly-time pre/post-processing. Intuition: **H is harder than E (or tied)**.

NP-hard: H is NP-hard if **for all** (E \in NP), (E \le_p H).
Intuition: H is harder than all NP problems (or tied).

NP-complete = (1) ∈ NP **and** (2) NP-hard
= not impossible + not easy = “hard” (Goldilocks).

An NP-complete problem is decidable, efficiently verifiable, and harder than every other NP problem (or tied).

14.3 A First NP-Complete Problem: Circuit Satisfaction (CSAT)

CSAT: input Boolean circuit C; output a satisfying assignment A, or None.

A **Boolean circuit** is a DAG whose vertices are AND, OR, NOT, OUT, or variables (x_i).
An **assignment** maps every variable to 0/1. A **satisfies** C if C outputs True on A. C is **unsatisfiable** if no assignment works.

Cook–Levin Theorem: CSAT is NP-complete.

Proof sketch:
1. **CSAT ∈ NP:** candidate = assignment A; verifier evaluates the circuit with A and accepts if OUT is True. Takes **O(n²)** poly time.
2. **CSAT is NP-hard:** for arbitrary (E \in NP), build reduction `solve_E`:
- compile `verify_E` into a Boolean circuit C (standard model: each CPU instruction / pseudocode step → circuit layers; poly layers × poly width = poly)
- `assignment = solve_CSAT(C)`

- convert assignment bits back into an E solution object
Pre/post-processing is polynomial ⇒ (E \le_p) CSAT for every E ∈ NP ⇒ CSAT is NP-hard.

Therefore CSAT is NP-complete.

14.4 Proving NP-Completeness by Reduction

(Understand the strategy — you will NOT be asked to prove an NP-complete reduction on the exam.)

Once one NP-complete problem exists, prove others by construction:

To prove H is NP-hard:
1. Pick a known NP-complete problem E
2. Design a reduction that solves E by calling a solver for H
3. Show pre/post-processing is polynomial
4. Conclude H is NP-hard (then also show H ∈ NP ⇒ NP-complete)

Notable NP-complete problems (know the names / definitions)

1. Circuit Satisfaction (CSAT)
2. Boolean Satisfaction (SAT)
3. Clique / Maximum Clique
4. Vertex Cover
5. Traveling Salesperson (TSP)
6. Set Partition
7. Many generalized games (Chess, Go, Sudoku, Mario, …)

Commonalities: in NP (verify fast); hard (exhaustive search / exponential seems necessary; too slow in practice); but useful if solved.

14.5 P Versus NP Revisited

We know **P ⊆ NP**. Open question: **P ⊂ NP** or **P = NP**? ($1M Clay prize.)

If P = NP	**If P ⊂ NP**
All NP (incl. NPC) get poly algorithms	NPC never have fast algorithms
New software features become practical	Case closed (like lower bounds)
Breaks cryptography	Crypto status quo
Proof strategy: find poly algo for one NPC	Proof strategy: prove no poly algo for every NPC (hard negative)

Conjecture / advice: live as if **P ⊂ NP** — treat NP-complete problems as impractical; treat crypto as secure. Expert poll majority agrees, but no consensus.

Categorize on the exam:
Class	Meaning	Example

P	poly-time solvable	Sorting, Edit Distance, Pair Sum, Fibonacci
NP-complete	in NP + NP-hard	CSAT, SAT, Clique, Vertex Cover, TSP
Undecidable	no algorithm exists	Halting Problem

Parallel Algorithms (Act III — bypassing limits)

Overcoming lower bounds

A tight sequential lower bound cannot be broken — but a **parallel** algorithm can make **user wait time (span)** smaller than the lower bound while **work** still respects it.

Analyzing parallel algorithms

Metric	Meaning	How to analyze
p	number of processors/cores	assume enough processors when useful (often p ~ n)
Work	total steps by all processors	ignore parallelism; count as usual
Span	elapsed time start→finish	take **max** of parallel branches; sequential steps as usual

Pseudocode:

```
in parallel:
  statement 1
  statement 2
  ...
```

Goal: work matches lower bound; span **beats** it.

Scalable: span O(X / p) for a problem with lower bound Ω(X).
Embarrassingly parallel: scalable with span **constant in n** (e.g. p-processor summation with p=n has span O(p)=O(1) w.r.t. n).

Parallel sorting

Sequential sorting: Θ(n log n) tight bound. Parallel sorting keeps **O(n log n) work** but achieves **sublinear span**.

Parallel merge sort

- Recurse on halves **in parallel**
- ~~**Parallel merge** itself is NOT on the exam~~ — know that with parallel merge, overall span is **O(log³ n)** and work **O(n log n)**
- First draft (parallel recurse, sequential merge): work O(n log n), span O(n) — merge is the bottleneck

Samplesort

Parallel cousin of quick sort:
1. Select **p−1 splitters** via oversampling (sample p·k elements, sort, take every k-th)
2. In parallel, bucket each element (binary search on splitters)
3. Recursively sample-sort buckets in parallel; concatenate

Algorithm	Work	Span
Parallel Merge Sort	O(n log n)	O(log³ n)
Samplesort	O(n log n) expected	**O(log n)** expected

Trade-off: samplesort has better span but expected time; parallel merge sort is deterministic.

Exam Performance Checklist

Tracing

- Merge sort: show every split and merge
- Out-of-place quick sort: show less/equal/greater and concatenations
- In-place partition / quick sort: show list after each swap / finished zones
- Edit distance / Fibonacci DP: fill the table cell-by-cell

Design patterns to practice

1. **Decrease-by-half:** base cases $n \leq 1$; divide; recurse; combine; Master Method analysis
2. **Reduction:** to sorting ($O(n \log n)$) or to hash maps ($O(n)$ expected)
3. **DP:** define subproblems, recurrence, base cases, fill table

Proof templates (write these on your notes page)

- **Master Method** 6 steps + 3 cases
- **$X \in P$:** exhibit poly-time algorithm
- **$X \in NP$:** define candidate + poly-time verifier
- **Trivial lower bound:** output size $\Rightarrow \Omega(\ldots)$
- **Sorting $\Omega(n \log n)$:** $n!$ perms, halve each comparison, $\log_2(n!) \in \Omega(n \log n)$
- **Halting undecidable:** contrary(contrary) contradiction

Common pitfalls

- Forgetting $n=1$ base case → infinite recursion
- Saying quick sort is $O(n \log n)$ without **expected** / forgetting **$O(n^2)$ worst-case**
- Confusing **work** vs **span**
- Confusing **P** (solvable fast) vs **NP** (verifiable fast) vs **undecidable**
- Trying to “break” a lower bound with a sequential algorithm
- Studying struck-through topics (American change, lower-bound reduction args, parallel merge details)

Suggested Study Order

1. Master Method + Merge Sort traces
2. Quick sort (both versions) traces + expected vs worst-case
3. Reduction recipes (sort vs hash) + PSQ times
4. Fibonacci + Edit Distance hand traces
5. Lower bounds: trivial + sorting tight bound
6. P / NP / Undecidable definitions + proof templates
7. Parallel: work/span + samplesort vs parallel merge sort table
8. Compress everything onto your **one-page notes** (see `ONE_PAGE_CHEAT_SHEET.md`)

Appendix: Worked Examples & Traces

These match the kinds of demos on the slides and the “trace by hand” exam prompts.

---

1. Merge Sort Trace

Input: `[2, 11, 1, 10]`

```
                  [2, 11, 1, 10]
                 /              \
            [2, 11]              [1, 10]
            /    \               /    \
         [2]    [11]          [1]    [10]
            \    /               \    /
         merge→[2,11]          merge→[1,10]
                \                  /
             merge→ [1, 2, 10, 11]
```

Merge detail for `[2,11]` and `[1,10]`:
- compare 2 vs 1 → take 1 → S=[1]
- compare 2 vs 10 → take 2 → S=[1,2]
- compare 11 vs 10 → take 10 → S=[1,2,10]
- leftover [11] → S=[1,2,10,11]

---

2. Out-of-Place Quick Sort Trace

Input: `[19, 9, 11, 5, 9, 2, 12]` — for practice, use **first element as pivot** each time (as exam demos often fix the random choice).

```
QS([19,9,11,5,9,2,12])  pivot=19
  less=[9,11,5,9,2,12]  equal=[19]  greater=[]
  QS([9,11,5,9,2,12])  pivot=9
    less=[5,2]  equal=[9,9]  greater=[11,12]
    QS([5,2]) pivot=5 → less=[2] equal=[5] greater=[] → [2,5]
    QS([11,12]) pivot=11 → less=[] equal=[11] greater=[12] → [11,12]
    combine → [2,5] + [9,9] + [11,12] = [2,5,9,9,11,12]
  combine → [2,5,9,9,11,12] + [19] + [] = [2,5,9,9,11,12,19]
```

---

3. In-Place Partition Trace

Input range: `[6, 7, 16, 4, 1]` with pivot chosen as front, then swapped to end (per slide demo style).

Suppose after moving pivot to end: list looks like `[1, 7, 16, 4, 6]` if pivot was 6…
Walk the i/j pointers:
- Grow `<pivot` when `U[i] < pivot`

- Grow ≥pivot when `U[j] ≥ pivot`
- Else swap `U[i], U[j]`
- Finally swap pivot into place between zones

After partition guarantee: everything left of p < pivot; everything right ≥ pivot.

---

4. Master Method Write-Up (exam style)

Claim: `merge_sort` takes O(n log n) time.

1. **t = 2** (general case when n ≥ 2)
2. **Base (n < 2):** T(n) = O(1)
3. **General:** T(n) = 2T(n/2) + 2n + 2 ∀ n ≥ 2
4. **Parts:** r=2≥1 ✓, d=2>1 ✓, t=2≥1 ✓, c(n)∈O(n) so k=1≥0 ✓
5. **r $\stackrel{?}{=}$ d^k:** 2 $\stackrel{?}{=}$ 2^1 → **equal → Case 2** → O(n^k log n)
6. **Simplify:** O(n^1 log n) = **O(n log n)**. QED

---

5. Fibonacci DP Hand Trace (slide demo: n = 5)

Invariant: A[i] = F(i)

i	**0**	**1**	**2**	**3**	**4**	**5**
A[i]	0	1	1	2	3	5

Output: A[5] = 5. Time O(n).

---

6. Edit Distance Hand Trace (slide demo: s="has", t="make")

Invariant: A[i][j] = distance between s[:i] and t[:j].
Ops: match → copy diagonal; else min(1+delete, 1+insert, 1+overwrite).

	ε	**m**	**a**	**k**	**e**
ε	0	1	2	3	4
h	1	1	2	3	4
a	2	2	1	2	3
s	3	3	2	2	**3**

Output: A[3][4] = 3. Time O(|s|×|t|)=O(n^2).

---

7. Hash Reduction (Set Intersection)

L=[5,19,1,3], R=[12,5,3,8]

Naïve: for each x in L, scan R → $O(n^2)$.

Hash:
1. Insert R → map {12,5,3,8}
2. Scan L: 5∈map ✓, 19✗, 1✗, 3✓ → output **[5,3]**
$O(n)$ expected.

---

8. Prove Pair Sum ∈ P (template)

```
naïve_pair_sum(L,k):
  for x in L:
    for y in L:
      if x+y==k: return (x,y)
  return None
```

$O(n^2)$ polynomial ⇒ **Pair Sum ∈ P**. QED

---

9. Prove Clique ∈ NP (template)

Candidate: vertex set C.
Verifier: check |C|=k; for all pairs in C, edge exists.
$O(n^3)$ (or better) polynomial ⇒ **Clique ∈ NP**. QED

---

10. Sorting Lower Bound (short form)

n! permutations. Each comparison halves candidates. Need $\log_2(n!)$ comparisons.
$\log_2(n!)=\Sigma_i \log_2 i$ has n terms $\Omega(\log n)$ each ⇒ **$\Omega(n \log n)$**.
Applies to every comparison-based sort. + merge sort $O(n \log n)$ ⇒ **$\Theta(n \log n)$** tight.

---

11. Parallel work vs span (2-proc sum)

```
in parallel:
  l = sum_range(0, mid)   # n/2
  r = sum_range(mid, n)   # n/2
return l+r
```

- Work ≈ (n/2)+(n/2) = **$\Theta(n)$** (matches $\Omega(n)$ LB)
- Span ≈ **n/2** (faster user wait)

That is how parallel “bypasses” a sequential lower bound without contradicting it.

Appendix: One-Page Cheat Sheet (for allowed notes)

SIDE A — Algorithms & Design Patterns

Master Method (decrease-by-half)

T(n)=r·T(n/d)+c(n) ∀n≥t; T(n)∈O(1) ∀n1; k≥0.
Compare $\mathbf{r \stackrel{?}{=} d^k}$: (1) rd^k → O(n^{log_d r})
Steps: t → base → general count → ID r,d,t,k → case → conclude.
Merge sort: 2T(n/2)+O(n) → case2 → **O(n log n)**. Binary search: T(n/2)+O(1) → **O(log n)**.

Merge Sort

`if n≤1: return U; half=n/2; return merge(MS(left),MS(right))`
merge: two-finger scan; leftovers append → **O(n)**. Trace: show every split + merge.

Quick Sort (Las Vegas: always correct; random runtime)

Out-of-place: `pivot=random_choice(U); L/E/G lists; return QS(L)+E+QS(G)`
In-place: partition zones < | todo | ≥ | pivot@end; final swap; recurse both sides.
O(n log n) expected, O(n²) worst. Extreme pivots ⇒ T(n)=T(n−1)+O(n). Prefer QS; if expected bad → Merge Sort; never O(n²) sorts.

Reduction

To algorithm: preprocess → solve B → postprocess. A reduces to B ⇒ A easier/tied.
To sorting: sort then scan → usually **O(n log n) exp.** Median: sort, pick mid. Setn: sort(L+R), take adjacent dupes.
To hash: naïve O(n²) bottleneck (search/count in loop) → HashMap O(1) exp → **O(n) expected**. Setn: put R in map; scan L. Mode: count_map then max.

Hash / PSQ

Hash ops critical: **O(1) expected** (collisions + chaining; pigeonhole ⇒ collisions inevitable).
PSQ: Insert O(1), RemoveMin O(log n) amort., DecreasePriority O(1) amort.
Prim–Jarník & Dijkstra + PSQ: **O(m + n log n)** (else O(mn)).

Dynamic Programming (tabulation; bottom-up)

Design: dimensions → **table invariant** → base → general → fill table.
Fib (1D): A[i]=F(i); A[0]=0,A[1]=1; A[i]=A[i−1]+A[i−2] → **O(n)**.
Edit distance (2D): A[i][j]=dist(s[:i],t[:j]). Base A[i][0]=i, A[0][j]=j.
If s[i−1]==t[j−1]: A[i−1][j−1]; else min(1+del A[i−1][j], 1+ins A[i][j−1], 1+ow A[i−1][j−1]).
Return A[|s|][|t|] → **O(|s|×|t|)=O(n²)**. Trace grids (ex: has→make).
~~American Change Making NOT ON EXAM~~

Parallel (Act III)

p=#processors. **Work**=total steps (ignore parallel). **Span**=elapsed (max of parallel).
Goal: work respects lower bound; span beats it. Embarrassingly parallel: span const in n.
Parallel merge sort: work **O(n log n)**, span **O(log³ n)**. ~~parallel merge NOT on exam~~
Samplesort: p−1 splitters (oversample), bucket in parallel, recurse; work **O(n log n) exp**, span **O(log n) exp**.

SIDE B — Limits, Classes, Proof Templates

Complexity cheat table

	MergeSort	QuickSort	Hash	Fib DP	EditDist	Prim/ Dijk+PSQ	Par.MS	Samplesort
Time	O(n log n)	O(n log n) exp / $O(n^2)$ wc	O(1) exp	O(n)	$O(n^2)$	O(m+n log n)	work O(n log n) span $O(\log^3 n)$	same work; span O(log n) exp

Lower bounds (Ch 12)

Upper (problem): exhibit algorithm. **Lower Ω**: EVERY algorithm ≥ that. **Tight Θ**: match ⇒ solved science.
Trivial LB: must create output of size f(n) ⇒ Ω(f(n)). Merge: Ω(n)+O(n)=**Θ(n)**.
Sorting: trivial Ω(n) leaves gap. Info-theoretic: n! perms; each cmp halves; need $\log_2(n!)=\Sigma \log i \in$ **Ω(n log n)** for comparison-based. + merge sort O ⇒ **Θ(n log n)** tight. ~~12.6 reduction LB proofs NOT ON EXAM~~

P / NP / Undecidable (Ch 13–14)

P = poly-time solvable ($O(n^k)$; includes n log n). **NP** = poly-time **verifiable** (exhaustive + poly verifier). **NP-complete** = in NP + NP-hard. **Undecidable** = no algorithm exists.
P ⊆ NP ⊆ Decidable; Undecidable outside. **P=NP?** open. If one NPC in P ⇒ all NP in P.
Halting: input (proc,I); True iff halts. Undecidable: assume halts → self_halts → contrary → contrary(contrary) contradiction.

Proof templates (copy onto exam answers)

∈ P: “Algorithm ___ solves X in O(**) poly time. QED.”**
∈ NP: “Candidate = . Verifier: **. Takes O(**) poly time ⇒ X∈NP. QED.”
Trivial LB: “Any algo must create output of size ___ ⇒ Ω(___).”
Sorting Ω(n log n): “n! perms; cmp halves candidates; $\log_2(n!)\in\Omega(n \log n)$; applies to all comparison sorts.”
Halting: “Suppose halts exists; build contrary; contrary(contrary) both cases contradict; no such algo.”

Design / categorize tips

Decrease-by-half: base $n \leq 1$ (avoid ∞ recursion); divide; recurse both; combine; Master Method.
Reduction: ask “does sorting or a hash map do the hard part?”
DP: define E(i)/E(i,j); recurrence; base; fill bottom-up; read answer.
Categorize: poly algo exists? → **P**. Only verify fast / NPC famous? → **NP-complete**. Literally impossible? → **undecidable** (Halting).
Will NOT ask: NP-complete reduction proofs. Bring **CWID**. Name on notes.